Both are viable options. Which strategy you adopt is up to your project requirements and your organization's structure. For the Polar Bookshop project, we will use the second option.

In a new `com.polarbookshop.orderservice.book` package, create a `Book` record to be used as a DTO, and include only the fields used by the order processing logic. As I pointed out earlier, in a real scenario I would expose a new endpoint in Catalog Service, returning book objects modeled as this DTO. For simplicity, we'll use the existing `/books/{bookIsbn}` endpoint, so any information that doesn't map to any of the fields in this class will be discarded when deserializing the JSON received into the Java object. Make sure the fields you do define have the same names as in the `Book` object defined in Catalog Service, or else the parsing will fail. That is something that consumer-driven contract tests can verify for you automatically.

Listing 8.19 The `Book` record is a DTO for storing book information

```
package com.polarbookshop.orderservice.book;

public record Book(
  String isbn,
  String title,
  String author,
  Double price
){}
```

Now that you have a DTO in Order Service ready to hold book information, let's see how you can retrieve it from Catalog Service.

8.3.3 Implementing REST clients with WebClient

The modern and reactive choice for REST clients in Spring is `WebClient`. The framework provides several ways of instantiating a